

Projeto 1 — Processamento de Imagens em C com SDL3

Etapa 1 — Análise inicial

Universidade Presbiteriana Mackenzie — Faculdade de Computação e Informática **Curso:** Ciência da Computação · **Disciplina:** Computação Visual **Professor:** André Kishimoto · **Semestre:** 2026.2 **Data de elaboração:** 08/09/2026

1. Integrantes do grupo

Nome completo	RA
João Guilherme Messias de Oliveira Santos	10426110

Observação sobre a composição do grupo. O enunciado exige grupos de **2 a 4 pessoas**, e a Tabela 3 de critérios prevê que o projeto seja zerado caso essa condição não seja atendida. No momento da elaboração desta análise inicial o grupo conta apenas com o integrante acima. A formação do grupo definitivo será regularizada junto ao professor antes da Etapa 2, e este relatório será reapresentado com os demais integrantes caso haja alteração.

2. Versões mais recentes e estáveis: o que muda em relação ao material da disciplina

O enunciado determina que o grupo use **as versões mais recentes e estáveis de todas as bibliotecas SDL**, e trata o material em vídeo como um *onboarding* possivelmente defasado. O levantamento abaixo foi feito consultando diretamente as páginas oficiais de *releases* dos repositórios `libsdl-org` em **08/09/2026**.

2.1 Versões-alvo adotadas pelo grupo

Biblioteca	Versão estável mais recente	Data do release
SDL (SDL3)	<code>release-3.4.16</code>	02/09/2026
SDL_image (SDL3_image)	<code>release-3.4.6</code>	02/09/2026
SDL_ttf (SDL3_ttf)	<code>release-3.2.2</code>	31/03/2025

Notas de leitura da tabela:

- SDL_image** possui duas linhas ativas mantidas em paralelo (`2.8.x` para SDL2 e `3.4.x` para SDL3). O projeto obriga SDL3, portanto a linha correta é a **3.4.x** — pegar a `2.8.12` por engano, por ser

numericamente "menor mas mais recente na listagem", é um erro fácil de cometer e que quebraria a compilação.

- **SDL_ttf** é a biblioteca com o ciclo de releases mais lento das três. A `3.2.2` é a versão estável mais recente da linha SDL3; as builds posteriores disponíveis são *prereleases* e, por não serem estáveis, foram descartadas.
- **SDL_ttf não é obrigatória** pelo enunciado (é apenas "sugestão" no item 4), mas o grupo pretende usá-la, porque os itens 4, 5 e 6 exigem texto em tela (informações do histograma e rótulos dos botões que mudam conforme o estado).

2.2 Principais diferenças esperadas em relação ao código-base

Como o código-base foi escrito por outra pessoa e o material de treinamento é anterior às versões acima, o grupo espera precisar ajustar os seguintes pontos. A lista foi montada a partir da leitura do guia de migração e das páginas de API no site oficial da SDL (wiki.libsdl.org).

a) Convenção de retorno das funções Em SDL2 a maioria das funções retornava `int` (`0` = sucesso, `-1` = erro). Em SDL3 elas retornam `bool` (`true` = sucesso). Todo `if (SDL_Func(...) < 0)` herdado do código-base precisa virar `if (!SDL_Func(...))`. É uma mudança silenciosa e perigosa: o código compila, mas a checagem de erro passa a estar invertida ou sempre falsa.

b) Criação de janela e renderer `SDL_CreateWindow()` deixou de receber as coordenadas `x / y` — a assinatura passou a ser `(título, largura, altura, flags)` e o posicionamento é feito depois, com `SDL_SetWindowPosition()`. `SDL_CreateRenderer()` também mudou: não recebe mais o índice do driver nem as flags, apenas `(janela, nome_driver)`. Isso afeta diretamente o requisito 3 (janela inicial 1024x768 centralizada) e o requisito 6 (redimensionar e reposicionar a janela principal).

c) Janela filha (requisito 3) A janela secundária precisa ser **filha** da principal. Em SDL3 isso não é uma flag simples de `SDL_CreateWindow()`: é preciso usar `SDL_CreateWindowWithProperties()` com a propriedade de janela-pai, ou associar o pai depois da criação. Esse é um dos pontos que o grupo julga menos coberto pelo material introdutório.

d) Desenho de texturas `SDL_RenderCopy()` foi substituída por `SDL_RenderTexture()`, e os retângulos de origem/destino passaram a ser `SDL_FRect` (ponto flutuante) em vez de `SDL_Rect` (inteiro). O mesmo vale para as primitivas de desenho usadas nos botões (`SDL_RenderFillRect` agora opera sobre `SDL_FRect`).

e) Acesso aos pixels da superfície O campo `format` de `SDL_Surface` deixou de ser um ponteiro para uma struct e passou a ser um valor do enum `SDL_PixelFormat`. Para descobrir o número de bytes por pixel, as máscaras e os deslocamentos de canal — necessários para a conversão para escala de cinza (requisito 2) e para o cálculo do histograma (requisito 4) — é preciso chamar `SDL_GetPixelFormatDetails()`. `SDL_MapRGB()` e `SDL_GetRGB()` também mudaram de assinatura por causa disso. Como alternativa mais simples e mais robusta, o grupo pretende **converter a superfície carregada para um formato conhecido e fixo** (por exemplo `SDL_PIXELFORMAT_RGBA32`) logo após o carregamento, com `SDL_ConvertSurface()`, e trabalhar sempre sobre esse formato. Isso elimina a necessidade de tratar N formatos diferentes de entrada.

f) Inicialização do SDL_image Em SDL2_image era necessário chamar `IMG_Init()` / `IMG_Quit()` com as flags dos formatos desejados. Em SDL3_image essas funções foram removidas: basta chamar

`IMG_Load()`. Código-base herdado que ainda chame `IMG_Init()` não compila.

g) Assinaturas do SDL_ttf 3 As funções de renderização de texto passaram a receber explicitamente o **comprimento da string** (`size_t length`, com `0` significando "string terminada em nulo"), e `TTF_OpenFont()` passou a receber o tamanho da fonte como `float`. É uma quebra de compatibilidade direta com qualquer exemplo escrito para SDL2_ttf.

h) Estrutura do `main` / ciclo de vida da aplicação SDL3 introduziu as *main callbacks* (`SDL_AppInit`, `SDL_AppIterate`, `SDL_AppEvent`, `SDL_AppQuit`) como alternativa ao `main()` com laço de eventos tradicional. **O grupo optou por manter o `main()` clássico com laço próprio**, porque o programa precisa receber o caminho da imagem por `argv` e porque o controle explícito do laço facilita o tratamento dos eventos de duas janelas distintas (requisitos 3, 5 e 6). Essa decisão será revista se o código-base escolhido já vier estruturado com os callbacks.

i) Versão do compilador O enunciado informa que o projeto será compilado com **gcc 15.1.0 no Windows e gcc 15.2.0 no WSL Ubuntu 26.04**. A máquina do grupo tem hoje **gcc 14.2.0** (MinGW-W64, UCRT, x86_64-ucrt-posix-seh, r3). Portanto, **a atualização do toolchain para o gcc 15.1.0 é a primeira tarefa técnica do projeto**, antes de qualquer linha de código. A distribuição escolhida será a build UCRT do WinLibs, para manter a compatibilidade com os pacotes `devel-mingw` oficiais da SDL, que são compilados contra a UCRT.

3. Processo de build

3.1 O processo do grupo é o mesmo do material da disciplina?

Provavelmente não, e essa é uma diferença deliberada. O material publicado pelo professor sobre configuração de projetos SDL3 parte de um fluxo baseado em **Visual Studio + MSBuild**, com a SDL adicionada ao projeto como *git submodule* e compilada a partir do código-fonte. Já o enunciado deste projeto determina explicitamente que **o código deve ser compilável com gcc**, o que torna o fluxo do Visual Studio/MSVC inadequado como processo principal.

Por isso o grupo adotará um processo próprio, descrito a seguir. Este item será reconferido após o grupo assistir integralmente aos vídeos da disciplina; caso o material já apresente um fluxo baseado em gcc, a diferença se reduzirá apenas às versões das bibliotecas e do compilador.

3.2 Processo adotado pelo grupo

1. **Toolchain.** MinGW-w64 (WinLibs, runtime **UCRT**, threads POSIX, exceções SEH), gcc **15.1.0**, instalado em `C:\mingw64` e adicionado ao `PATH`.
2. **Bibliotecas.** Uso dos pacotes **oficiais pré-compilados para MinGW** (`SDL3-devel-<versão>-mingw.zip`, `SDL3_image-devel-<versão>-mingw.zip`, `SDL3_ttf-devel-<versão>-mingw.zip`), extraídos para uma pasta `libs/` fora do controle de versão. A opção pelos binários prontos, em vez de compilar a SDL do zero via CMake, foi feita para reduzir o número de variáveis do ambiente e tornar o build reproduzível mais rapidamente — compilar a SDL a partir do submodule permanece como plano B.

3. **Build.** Um `Makefile` único na raiz do projeto, invocado por `mingw32-make`, compilando com `gcc -std=c17 -Wall -Wextra` e ligando com `-lSDL3 -lSDL3_image -lSDL3_ttf`. O `Makefile` é o contrato de build: qualquer pessoa consegue compilar o projeto com um único comando, sem depender do editor.
4. **Editor.** VS Code, com um `tasks.json` que apenas dispara o `Makefile` (nunca duplicando os comandos de compilação) e um `c_cpp_properties.json` para o IntelliSense enxergar os headers da SDL.
5. **Distribuição.** As DLLs da SDL (`SDL3.dll`, `SDL3_image.dll`, `SDL3_ttf.dll`) são copiadas para junto do executável como passo do `Makefile`, já que no Windows elas precisam estar no mesmo diretório do binário.
6. **Higiene do repositório.** Um `.gitignore` exclui a pasta de build, os objetos `.o`, o executável, as DLLs e a pasta `libs/`. Isso atende diretamente ao critério da Tabela 3 que penaliza em **-1,0 ponto** o envio de arquivos e pastas intermediárias do processo de compilação.

3.3 Verificação cruzada no WSL

Como o enunciado informa que o projeto **também será compilado no WSL Ubuntu 26.04 com gcc 15.2.0**, o grupo pretende validar periodicamente a compilação nesse ambiente, instalando as bibliotecas de desenvolvimento via gerenciador de pacotes e usando o mesmo `Makefile` (com a detecção de plataforma tratando as diferenças de extensão do executável e de flags de link). Um projeto que compila só no Windows corre o risco de ser zerado pelo critério "projeto não compila".

4. Sistemas operacionais, compiladores e editores utilizados

O grupo utilizará **uma única configuração** como ambiente principal de desenvolvimento:

Item	Configuração	Versão verificada
Sistema operacional	Windows 11 Pro	10.0.26200
Compilador (atual)	gcc — MinGW-W64 x86_64-ucrt-posix-seh	14.2.0
Compilador (alvo)	gcc — MinGW-w64 UCRT (WinLibs)	15.1.0
Editor	Visual Studio Code	1.135.0
Controle de versão	Git para Windows	2.53.0.windows.2
Ambiente secundário (validação)	WSL — Ubuntu 26.04 + gcc	15.2.0

O WSL não é um segundo ambiente de desenvolvimento, e sim um **ambiente de verificação**: o desenvolvimento acontece no Windows, e o WSL serve apenas para confirmar que o código compila e roda na mesma configuração que o professor usará.

5. Potenciais problemas decorrentes de configurações diferentes

O enunciado permite desconsiderar este item quando todos os integrantes usam a mesma configuração, o que é o caso atual. Ainda assim, o grupo identificou riscos reais de portabilidade que valem registro, porque o projeto **será avaliado em duas plataformas diferentes daquela em que é desenvolvido**:

1. **Diferença de versão do gcc (14.2.0 local × 15.1.0 da avaliação).** O gcc 15 tornou padrão o modo C23 e endureceu diagnósticos que antes eram apenas avisos — protótipos implícitos e conversões implícitas de ponteiro, por exemplo, passaram a ser erros. Código que compila localmente hoje pode simplesmente não compilar na máquina do professor. Mitigação: fixar `-std=c17` (ou `-std=c23`) explicitamente no `Makefile` e compilar com `-Wall -Wextra`, tratando avisos como problemas a resolver e não a ignorar.
2. **Caminhos de arquivo e separadores.** O caminho da imagem chega por `argv`, e o Windows usa `\` enquanto o Linux usa `/`. O caminho da fonte usada nos textos (requisito 8) é o caso mais crítico: a fonte precisa ser carregada corretamente **independentemente do sistema operacional**. Mitigação: distribuir o arquivo da fonte junto do projeto (em `assets/`) e resolver o caminho em tempo de execução a partir do diretório do executável, com `SDL_GetBasePath()`, em vez de depender de fontes instaladas no sistema (`C:\Windows\Fonts` ou `/usr/share/fonts`), que não existem nos dois lados.
3. **Fim de linha (CRLF × LF).** Editar no Windows e compilar no WSL com o mesmo repositório costuma produzir diffs de arquivos inteiros por causa da mudança de fim de linha. Mitigação: um `.gitattributes` normalizando os arquivos-fonte.
4. **Sensibilidade a maiúsculas nos `#include`.** O sistema de arquivos do Windows não diferencia maiúsculas de minúsculas; o do Linux, sim. Um `#include <SDL3/sdl.h>` compila no Windows e falha no WSL. Mitigação: conferir a grafia exata dos headers.
5. **Comportamento de janelas entre Windows e Linux.** O requisito 3 (janela filha) e o requisito 6 (janela centralizada, ou em (0,0) quando maior que a tela) dependem do gerenciador de janelas. O comportamento de janelas-pai e de posicionamento explícito difere entre o backend do Windows e o Wayland/X11. Este é o requisito com maior risco de "funciona aqui, não funciona lá".

6. Itens que o grupo consegue resolver apenas com o material da disciplina e a documentação da SDL

A tabela abaixo classifica cada item do "Escopo e funcionalidades obrigatórias" em três níveis de conforto, conforme pedido pelo enunciado.

#	Item do escopo	Nível	Justificativa
1	Carregamento de imagem	Confortável	<code>IMG_Load()</code> seguido de checagem de retorno nulo e <code>SDL_GetError()</code> . A documentação do SDL_image cobre o caso por completo, e o tratamento de erro é padrão de C.

#	Item do escopo	Nível	Justificativa
2	Análise e conversão para escala de cinza	Confortável	A fórmula $Y = 0.2125 \cdot R + 0.7154 \cdot G + 0.0721 \cdot B$ é dada pelo enunciado. Detectar se a imagem é colorida é percorrer os pixels e verificar se $R == G == B$ em todos eles. O trabalho está em ler os pixels corretamente, não no algoritmo. Já explorei transformações ponto a ponto (negativo e alargamento de contraste) nas atividades do blog da disciplina, o que cobre a parte conceitual.
3	GUI — janela principal 1024x768 centralizada	Confortável	<code>SDL_CreateWindow()</code> + <code>SDL_SetWindowPosition()</code> com <code>SDL_WINDOWPOS_CENTERED</code> . Documentado.
3	GUI — janela secundária filha , em (0,0)	Preciso pesquisar	Ver seção 7.1.
4	Cálculo do histograma	Confortável	Um vetor de 256 posições e uma passada pelos pixels. É contagem de frequência.
4	Desenho do histograma "de forma clara e proporcional"	Dificuldade parcial	Desenhar as 256 barras é direto com <code>SDL_RenderFillRect</code> . A parte não trivial é a normalização : um único pico dominante (um fundo uniforme, por exemplo) achata visualmente todo o resto. Preciso decidir entre normalizar pelo máximo, cortar percentis ou usar escala logarítmica — decisão de visualização, não de API.
4	Média e desvio padrão, com classificação	Confortável	Média e desvio padrão calculados diretamente do histograma. O que exige critério é onde cortar as faixas "clara/média/escura" e "alto/médio/baixo contraste": os limiares serão definidos e justificados pelo grupo no relatório final.
5	Botões desenhados com primitivas e com estados visuais	Confortável	Um retângulo, um teste de colisão ponto-retângulo com a posição do mouse e três cores conforme o estado (neutro / mouse em cima / pressionado). Não requer nada além de <code>SDL_RenderFillRect</code> e dos eventos de mouse.
5	Alternar entre imagem equalizada e original sem recarregar	Confortável	Manter duas cópias em memória (a versão em cinza original e a equalizada) e trocar o ponteiro exibido. É gerenciamento de memória, que é conteúdo de C.
5	Algoritmo de equalização do histograma	Preciso pesquisar	Ver seção 7.2.
6	Alternar resolução original × 1024x768 e redimensionar a janela	Dificuldade parcial	<code>SDL_SetWindowSize()</code> e <code>SDL_RenderTexture()</code> com destino escalado resolvem o essencial. A regra condicional ("centralizada, exceto se exceder a resolução do sistema — aí em (0,0)") exige consultar o modo de vídeo do monitor principal, o que já é menos direto. Ver seção 7.4.

#	Item do escopo	Nível	Justificativa
7	Salvar imagem com a tecla <code>S</code>	Confortável	<code>IMG_SavePNG()</code> e um evento de teclado. Sobrescrever é o comportamento padrão; detectar se o arquivo já existia (para escolher a mensagem "criado" × "sobrescrito") é feito antes de salvar, com uma checagem de existência do arquivo.
8	Carregar fonte de forma independente do SO	Dificuldade parcial	A API do <code>SDL_ttf</code> é simples; o problema é de estratégia de distribuição e licenciamento do asset , não de API. Ver seções 5 (item 2) e 7.5.

7. Itens que exigirão pesquisa mais aprofundada ou apoio de IA generativa

Conforme as regras da Etapa 1, esta análise foi elaborada **sem uso de IA generativa**, consultando apenas o material da disciplina e a documentação oficial da SDL. Os itens abaixo são aqueles em que o grupo reconhece que precisará ampliar as fontes de consulta na Etapa 2.

7.1 Janela secundária como janela filha (requisito 3)

Por quê. É o item em que o grupo tem menos base. A documentação da SDL descreve a propriedade de janela-pai em `SDL_CreateWindowWithProperties()`, mas não deixa claro o comportamento resultante em cada plataforma: se a janela filha sempre fica acima da principal, se aparece na barra de tarefas, se é fechada junto com a principal e como isso muda entre Windows e Linux. Além disso, o tratamento de eventos com duas janelas exige distinguir a origem de cada evento pelo `windowID`, o que não aparece nos exemplos introdutórios de janela única.

7.2 Equalização de histograma (requisito 5)

Por quê. Este é o item de maior peso na avaliação do software (2,0 pontos, dos quais 1,5 só para a equalização estar correta) e é o único requisito que é de fato um algoritmo de processamento de imagens, e não uso de API. Conheço a ideia geral — redistribuir as intensidades usando a **função de distribuição acumulada (CDF)** do histograma para "espalhar" os tons pela faixa completa — mas ainda não implementei. Os pontos que preciso estudar com cuidado:

- a **fórmula exata** do mapeamento, incluindo o tratamento do valor mínimo não nulo da CDF (`cdf_min`), que evita que a imagem resultante fique deslocada;
- a aritmética inteira envolvida (arredondamento e risco de estouro ao multiplicar a CDF acumulada por 255 em imagens grandes);
- a diferença prática entre a equalização e o **alargamento de contraste** e a **correção gama**, que já estudei nas atividades da disciplina. As três atacam o mesmo problema, mas só a equalização usa a distribuição real da imagem — e entender essa diferença é o que me permite justificar o resultado obtido, em vez de apenas exibi-lo.

7.3 Manipulação de pixels em formatos arbitrários (requisitos 2, 4, 5 e 7)

Por quê. É a base silenciosa de quase todo o projeto: se a leitura dos pixels estiver errada, o tom de cinza, o histograma, a equalização e a imagem salva estarão todos errados juntos. As dúvidas concretas são o cálculo correto do endereço de um pixel usando o `pitch` (que não é igual a `largura × bytes por pixel`, por causa do alinhamento de linha), quando é necessário `SDL_LockSurface()` e a ordem dos canais em cada formato. A estratégia de converter tudo para um formato fixo logo no carregamento (seção 2.2, item "e") foi pensada justamente para reduzir esse risco, e precisa ser validada.

7.4 Regra de posicionamento condicional da janela (requisito 6)

Por quê. Exige consultar a resolução do monitor principal em tempo de execução e decidir entre centralizar ou fixar em (0,0). É um item pequeno, mas depende de APIs de display que não costumam aparecer em material introdutório e que tendem a se comportar de forma diferente em monitores com escala (DPI) alta — um detalhe que pode fazer a janela "parecer" descentralizada mesmo com a conta correta.

7.5 Escolha e empacotamento da fonte (requisito 8)

Por quê. A dificuldade não é técnica, e sim de decisão informada: escolher uma fonte com **licença que permita a redistribuição junto do código** em um repositório público. O grupo pretende usar uma fonte de licença aberta (família DejaVu ou Liberation, ambas redistribuíveis), incluí-la em `assets/fonts/` e carregá-la por caminho relativo ao executável. A verificação da licença exige consultar fontes externas à documentação da SDL.

8. Resumo da autoavaliação

Situação	Itens do escopo
Confortável — resolvo com o material da disciplina e a documentação da SDL	1 (carregamento), 2 (escala de cinza), 3 (janela principal), 4 (cálculo do histograma e estatísticas), 5 (botões e alternância sem recarregar), 7 (salvar imagem)
Dificuldade parcial — sei o caminho, faltam detalhes	4 (desenho proporcional do histograma), 6 (redimensionamento e regra de posicionamento), 8 (fonte independente de SO)
Preciso pesquisar a fundo	3 (janela filha), 5 (algoritmo de equalização), manipulação de pixels em formatos arbitrários

9. Próximos passos

1. Atualizar o toolchain para gcc 15.1.0 (WinLibs UCRT) e confirmar com `gcc --version`.
2. Baixar os pacotes `devel-mingw` da SDL3 3.4.16, SDL3_image 3.4.6 e SDL3_ttf 3.2.2.
3. Escolher o projeto de exemplo do repositório da disciplina que servirá de base.
4. Escrever o `Makefile` e validar um "hello, window" compilando e executando.

5. Validar o mesmo `Makefile` no WSL Ubuntu.
 6. Implementar os requisitos na ordem 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, com commits pequenos e frequentes, atendendo ao critério de "atualizações significativas e constantes" da Tabela 1.
-

10. Referências consultadas

- Enunciado do Projeto 1 (Proj1) — Computação Visual, Prof. André Kishimoto, 2026.2.
 - Vídeos e repositório de exemplos da disciplina (Moodle).
 - Documentação oficial da SDL — <https://wiki.libsdl.org/SDL3/>
 - Guia de migração SDL2 → SDL3 — <https://wiki.libsdl.org/SDL3/README-migration>
 - Páginas de *releases* oficiais: [libsdl-org/SDL](https://wiki.libsdl.org/SDL), [libsdl-org/SDL_image](https://wiki.libsdl.org/SDL_image) e [libsdl-org/SDL_ttf](https://wiki.libsdl.org/SDL_ttf) (consultadas em 08/09/2026).
-

Relatório elaborado sem uso de IA generativa, conforme as regras da Etapa 1.